Lab02：多线程编程

本目录是 Lab02 的提交内容，包含两个用 POSIX 线程（pthread）编写的 C 程序：

- **Part I**：`sudoku.c`，用 11 个工作线程检查一个 9×9 数独答案是否正确；
- **Part II**：`sort.c`，用 2 个排序线程和 1 个合并线程给一串整数排序。

文件说明

文件	内容
`sudoku.c`	Part I 源代码
`sort.c`	Part II 源代码
`README.md`	本文件：设计简介、编译和使用方法
`REPORT.md`	实验报告
`GUIDE-1.md`、`GUIDE-2.md`	`sudoku.c`、`sort.c` 的逐行讲解，面向初学者
`Lab02.md`	实验要求
`build/`	编译产物，由上面的命令生成，已被 `.gitignore` 忽略

运行环境

- Linux（内核 2.6 及以后）、GCC、glibc（自带 pthread 线程库）
- 已测试环境：Ubuntu 26.04.1 LTS（WSL2，内核 6.18）、GCC 15.2.0、glibc 2.43

编译

```
mkdir -p build && gcc -std=c99 -O2 -Wall -fno-asm "sudoku.c" -o "build/sudoku" --static -lm
mkdir -p build && gcc -std=c99 -O2 -Wall -fno-asm "sort.c" -o "build/sort" --static -lm
```

可执行文件生成在 `build/` 目录中，分别是 `build/sudoku` 和 `build/sort`。`build/` 已写入 `.gitignore`，构建产物不会提交。

选项说明：`-std=c99` 按 C99 标准编译，`-O2` 开启二级优化，`-Wall` 打开常用警告，`-fno-asm` 禁用 `asm` 关键字（程序本身没有使用内联汇编），`--static` 静态链接，`-lm` 链接数学库。两个程序编译均无警告；另外加上 `-Wextra -pedantic` 检查也没有警告。

Part I：数独答案校验（sudoku）

使用方法

程序从标准输入读入 81 个整数：数独按从上到下、每行从左到右的顺序排列，数与数之间用空格或换行分隔。例如把实验文档中的数独保存为 `puzzle.txt`：

```
6 2 4 5 3 9 1 8 7
5 1 9 7 2 8 6 3 4
8 3 7 6 1 4 2 9 5
1 4 3 8 6 5 7 2 9
9 5 8 2 4 7 3 6 1
7 6 2 3 9 1 4 5 8
3 7 1 9 5 6 8 4 2
4 9 6 1 8 2 5 7 3
2 8 5 4 7 3 9 1 6
```

运行：

```
./build/sudoku < puzzle.txt
```

输出：

```
Thread  0: valid    all rows
Thread  1: valid    all columns
Thread  2: valid    subgrid rows 1-3, columns 1-3
Thread  3: valid    subgrid rows 1-3, columns 4-6
Thread  4: valid    subgrid rows 1-3, columns 7-9
Thread  5: valid    subgrid rows 4-6, columns 1-3
Thread  6: valid    subgrid rows 4-6, columns 4-6
Thread  7: valid    subgrid rows 4-6, columns 7-9
Thread  8: valid    subgrid rows 7-9, columns 1-3
Thread  9: valid    subgrid rows 7-9, columns 4-6
Thread 10: valid    subgrid rows 7-9, columns 7-9
The Sudoku solution is valid.
```

前 11 行依次是每个工作线程的检查结果（`valid` 合格，`invalid` 不合格）和它负责的区域，最后一行是结论。

情况	输出	退出码
读入了 81 个整数	11 个线程的结果和最终结论（valid 或 invalid）	0
不足 81 个整数，或含有非整数	向标准错误输出 `Error: expected 81 integers on standard input`	1

设计简介

- **线程分工**：线程 0 检查全部 9 行，线程 1 检查全部 9 列，线程 2～10 各检查一个 3×3 子格（从左到右、从上到下）。
- **传递参数**：主线程用 `malloc` 申请一个 `parameters` 结构体，写入起始行 `row` 和起始列 `column`，通过 `pthread_create` 交给工作线程；工作线程用完后自己 `free`。
- **返回结果**：全局数组 `int result[11]`，线程 i 只写 `result[i]`（1 表示合格，0 表示不合格）。子格线程根据自己的起点算出下标：`2 + (row / 3) * 3 + column / 3`。
- **同步**：主线程用 `pthread_join` 等 11 个线程全部结束后，才读取 `result` 并打印。数独在创建线程之前就已读好，之后只读不写；各线程写的是 `result` 中不同的格子，因此不需要加锁。

Part II：多线程排序（sort）

使用方法

程序从标准输入读入任意多个（至少 1 个）整数，数与数之间用空格或换行分隔，一直读到输入结束：

```
echo 7 12 19 3 18 4 2 6 15 8 | ./build/sort
./build/sort < list.txt
```

输出：

```
Original list:    7 12 19 3 18 4 2 6 15 8
Sorting thread 0: 3 7 12 18 19
Sorting thread 1: 2 4 6 8 15
Sorted list:      2 3 4 6 7 8 12 15 18 19
```

四行依次是：原始列表、线程 0 排好的前一半、线程 1 排好的后一半、合并后的完整有序列表。

- 用键盘输入时，输完数字后换到新的一行，按 Ctrl+D 表示输入结束。
- 运行时要写 `./build/sort`；只写 `sort` 会运行系统自带的 `sort` 命令。

情况	输出	退出码
读入了至少 1 个整数	上面的 4 行结果	0
没有输入，或含有非整数（例如用逗号分隔）	向标准错误输出 `Error: expected integers separated by whitespace on standard input`	1

设计简介

- **线程分工**：列表从中间分成两半（个数为奇数时后一半多 1 个）。线程 0、1 各用插入排序把一半原地排好，线程 2 把排好的两半合并进第二个数组。
- **数据**：全局指针 `list` 指向读入的列表（读入时用 `realloc` 按需扩大），`sorted` 指向与 `list` 同样大小的第二个数组，全局变量 `count` 是元素个数。
- **传递参数**：与 Part I 相同，用 `malloc` 申请 `parameters` 结构体，写入线程负责的左闭右开范围 `[start, end)`，由工作线程用完后 `free`。
- **同步**：主线程先用 `pthread_join` 等两个排序线程都结束，再创建合并线程，等它也结束后才打印结果。两个排序线程只访问各自的一半，合并线程运行时排序线程已经结束，因此不需要加锁。

更多内容

- `REPORT.md`：实验报告，包括 Linux 线程控制方法总结、程序设计、实验结果与分析、遇到的问题及解决方法、参考资料、建议与意见。
- `GUIDE-1.md`、`GUIDE-2.md`：逐行代码讲解。